

Software Architecture and Design Specification

Project: Railway Reservation System

Version: 1.0

Authors: Team 2 – Ankit Uttarakar(PES2UG24CS070 ; Anishwar Rajaram(PES2UG24CS068) ; Ankirapalle Goel
Suhas(PES2UG24CS069) ; Ankit Verma(PES2UG24CS071)

Date: 03-10-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	03-10-2026	Team 2	Software Architecture and Design Specification for Railway Reservation System

Approvals

Role	Name	Signature/Date
Course Coordinator	Asha Kumari A	

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Railway Reservation System.

1.2 Scope

Covers passenger interactions for train search, schedule and seat-availability checking, ticket booking, passenger details, fare calculation, payment recording, PNR generation, ticket viewing, cancellation and refund processing, administrative functions, and operational record management.

1.3 Audience

Developers, QA Engineers, System Administrators, Railway Operators, and Assessment Evaluators.

1.4 Definitions

SRS, PNR, UI, API, DB, C++, FR, NFR, RTM.

2. Document Overview

2.1 How to use this document

Provides the architecture and design description for the Railway Reservation System, including the component diagram, component descriptions, architecture pattern, technology/data-store information, risks, traceability, security architecture, sequence diagrams, interfaces, error handling, and UX design.

2.2 Related Documents

SRS; Software Test Plan (STP); RTM.

3. Architecture

3.1 Goals & Constraints

- C/C++ implementation environment.
- Console / menu-driven user interface.
- Persistent storage for passenger, train, schedule, seat, booking, payment and cancellation records.
- Only authorized administrators may modify railway master data and reservation records.
- Data consistency shall be maintained between seat inventory and confirmed reservations.

3.2 Stakeholders & Concerns

Stakeholder	Concern
Passenger	Clear train information, simple booking/cancellation and quick confirmation.
Railway Staff / Administrator	Management of trains, schedules, fares, seat inventory and reservation records.
Assessment Evaluator	Verification of functional, non-functional and security requirements.

3.3 Component (UML) Diagram

Figure 1: Railway Reservation System Component Diagram

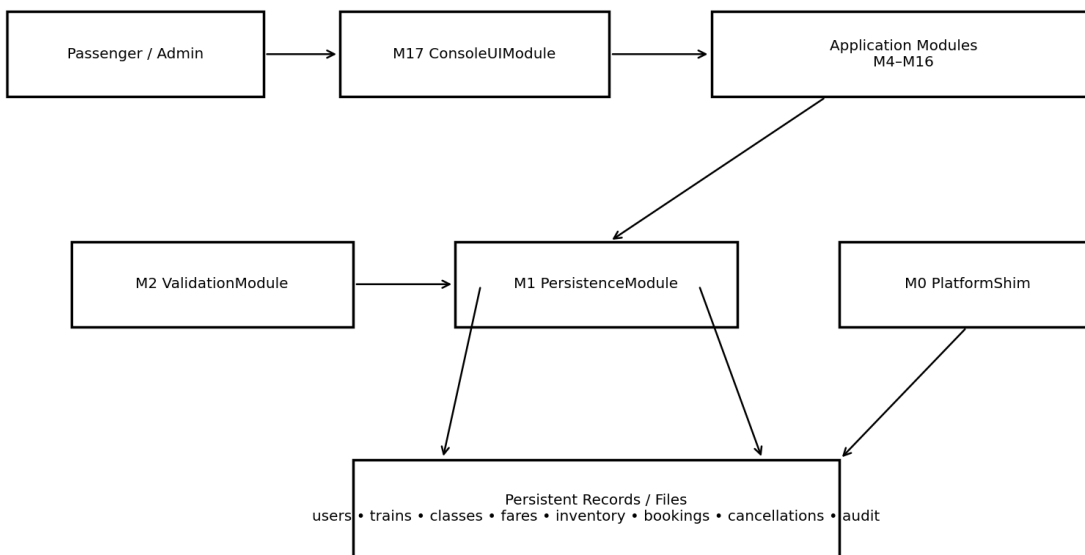


Figure 1: Component Diagram

3.4 Component Descriptions

Component / Module	Description
M0 PlatformShim	Platform-specific facilities used by the application.
M1 PersistenceModule	Persistent storage for project records.
M2 ValidationModule	Validation of user input and required values.
M3 AuditModule	Security-relevant and operational audit records.
M4 AuthenticationModule	Passenger/administrator authentication.
M5 AuthorizationModule	Role-based access control.
M6 TrainCatalogueModule	Train and schedule master data.
M7 ClassFareConfigModule	Travel-class and fare configuration.
M8 SeatInventoryModule	Seat/berth availability and allocation.
M9 SearchModule	Train search.
M10 PassengerModule	Passenger details.
M11 FareCalculationModule	Fare calculation.
M12 PaymentModule	Payment recording/outcome.
M13 BookingModule	Booking and seat allocation coordination.
M14 TicketRetrievalModule	Ticket/booking retrieval.
M15 CancellationRefundModule	Cancellation and refund processing.
M16 ReportingModule	Reservation and operational reports.
M17 ConsoleUIModule	Console user interface.

3.5 Chosen Architecture Pattern and Rationale

The system is a C/C++ application with a console interface, application modules, and persistent storage. The design separates the console UI, application modules and persistence responsibilities so that the reservation functions and persistent records can be managed and tested separately.

3.6 Technology Stack & Data Stores

Area	Specification
Language	C/C++17
Application interface	Console / menu-driven interface
Persistent storage	File or database interface for persistent reservation records, as specified by the SRS
Application logic	C/C++ standard libraries and application components
Testing	Catch2 v3 unit/component tests and CLI end-to-end flows

Reference environment	Linux/g++; Windows/MinGW secondary
-----------------------	------------------------------------

3.7 Risks & Mitigations

Risk	Mitigation
Inconsistent reservation records	Maintain consistency between seat inventory and confirmed reservations.
Unauthorized administration	Restrict administrator functions to authorized administrator accounts.
Invalid input	Validate user input before processing.
Connection failure where network communication is used	Handle connection failures and preserve transaction consistency.

3.8 Traceability to Requirements

Requirement	Architecture / Design element
RRS-F-001 / authentication	M4 AuthenticationModule
RRS-F-003 / administrator restriction	M5 AuthorizationModule
RRS-F-004–RRS-F-006 / train search and availability	M6 TrainCatalogueModule, M8 SeatInventoryModule, M9 SearchModule
Booking / fare / payment	M10 PassengerModule, M11 FareCalculationModule, M12 PaymentModule, M13 BookingModule
Ticket retrieval / cancellation / refund	M14 TicketRetrievalModule, M15 CancellationRefundModule
Reports / audit	M16 ReportingModule, M3 AuditModule
Security requirements	M4 AuthenticationModule, M5 AuthorizationModule, M2 ValidationModule, M3 AuditModule, M1 PersistenceModule

3.9 Security Architecture

Security objectives: (1) protect passenger, booking, and payment-related data from unauthorized access or disclosure; and (2) prevent unauthorized modification of train, schedule, seat, booking, payment, and cancellation records.

Security requirement	Design element
RRS-SR-001: authentication before protected functions	M4 AuthenticationModule
RRS-SR-002: role-based access for administrator operations	M5 AuthorizationModule
RRS-SR-003: validate all user input	M2 ValidationModule
RRS-SR-004: protect stored credentials; no plaintext passwords	Authentication/persistence design
RRS-SR-005: record security-relevant actions	M3 AuditModule

4. Design

4.1 Design Overview

The system uses a console-based user interface and application components for reservation operations. Persistent reservation records are accessed through the storage interface.

4.2 UML Sequence Diagrams

Figure 2: Booking Flow

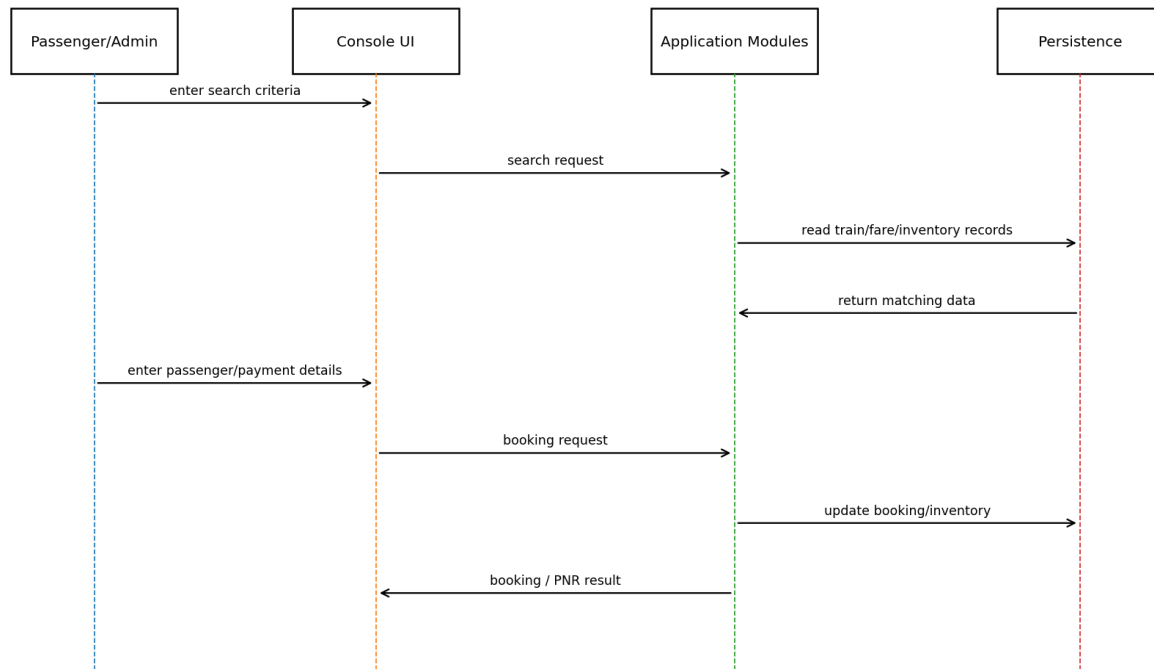


Figure 2: Booking Sequence Diagram

Figure 3: Cancellation / Refund Flow

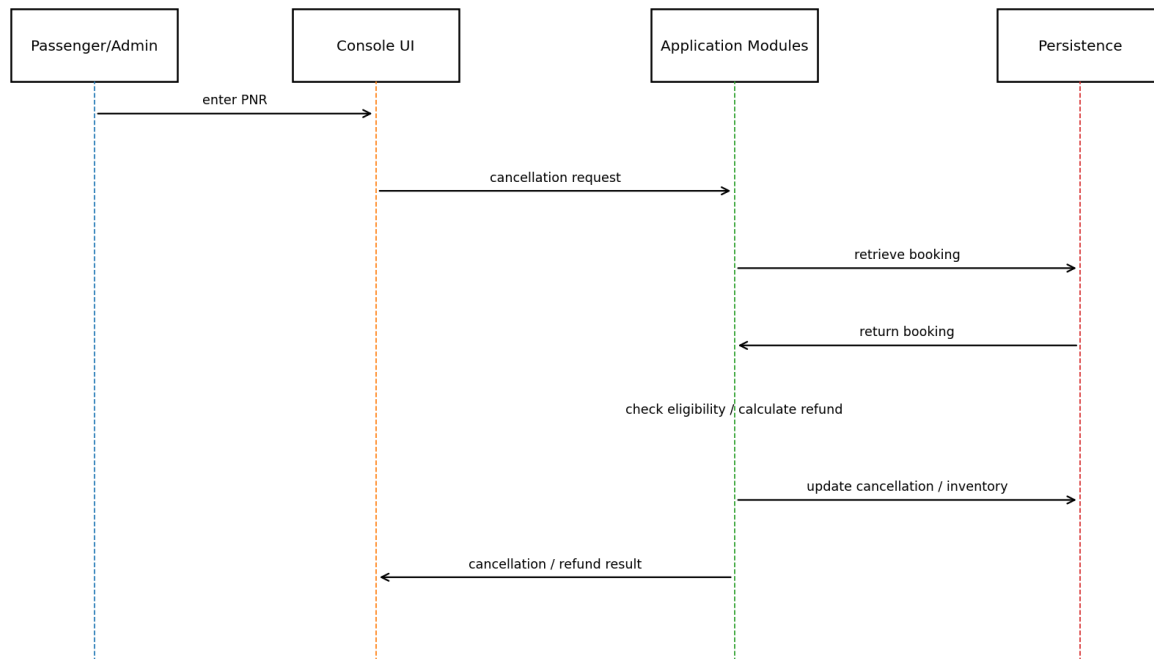


Figure 3: Cancellation / Refund Sequence Diagram

4.3 API Design

The project is specified as a console-based C/C++ application. No external REST API endpoints are specified in the SRS/Test Plan. The component interfaces are therefore internal application interfaces.

Component interface	Purpose
AuthenticationModule	Authenticates passenger or administrator credentials before protected functions.
AuthorizationModule	Checks whether the authenticated role may perform an operation.
PersistenceModule	Provides the application components with access to persistent reservation records.

4.4 Error Handling, Logging & Monitoring

Messages shall clearly indicate success, failure, unavailable seats, and invalid input. Failed operations shall not leave partially completed booking or cancellation records. Security-relevant actions such as login failures, cancellations and administrative changes shall be recorded in the audit log. The Test Plan covers performance, reliability, robustness, portability and audit logging.

4.5 UX Design

Primary UI: clear menu-driven interface for passenger and administrator operations. The interface shall display train search results, availability, passenger details, booking confirmation, PNR, cancellation status, and messages for invalid input.

Passenger: registration/login, train search, seat availability, booking, ticket viewing and cancellation.

Administrator: management of trains, schedules, fares, seat inventory and reservation records, plus reporting/monitoring functions.

4.6 Open Issues & Next Steps

The provided SAD template does not specify additional project-specific open issues. Implementation details not fixed by the SRS/Test Plan should be finalized during implementation without changing the stated requirements.

5. Appendices

5.1 Glossary

Term	Meaning
ATM	Not applicable to this project; retained only as a template example term.
RRS	Railway Reservation System
PNR	Passenger Name Record
SRS	Software Requirements Specification
STP	Software Test Plan
RTM	Requirements Traceability Matrix
C++	Implementation language specified for the project

5.2 References

Team 2 Railway Reservation System SRS v2.0 (03-09-2026); Team 2 Railway Reservation System Test Plan v1.0 (16-09-2026); supplied Software Architecture and Design Specification template.

5.3 Tools

C/C++17; g++ / MinGW; Catch2 v3; CMake; GitHub Actions; draw.io / UML diagramming tools where required.